

Proyecto Final MITEX: CulinaryRAG

Sistema de soporte inteligente y recomendación conversacional de recetas

Curso 2025/2026

Índice

1. Introducción y Objetivos	3
2. Diseño Arquitectónico Global	3
2.1. Pipeline Offline	4
2.2. Pipeline Online	4
3. Matriz de Cumplimiento del Enunciado	4
4. Detalles de Implementación	5
4.1. Ingestión y Limpieza	5
4.2. Recuperación Híbrida	5
4.3. Modelado de Tópicos	5
4.4. Resumen Extractivo	5
4.5. LLM y Generación	6
5. Validación y Discusión	6
5.1. Validación por Módulos	6
5.2. Experimento de Tópicos	6
5.3. Limitaciones	6
6. Instrucciones de Ejecución	7
7. Conclusiones y Trabajo Futuro	7

1. Introducción y Objetivos

Este informe documenta el diseño, implementación y validación de **CulinaryRAG**, un asistente conversacional especializado en el dominio culinario. El sistema recomienda recetas a partir de consultas en lenguaje natural, resume reseñas asociadas y puede activar una búsqueda web básica cuando la pregunta queda fuera de la base local.

El corpus procede del dataset público *Food.com Recsys Dataset*. Tras el preprocesado, el proyecto trabaja con recetas, ingredientes, pasos, etiquetas, tiempos de preparación y reseñas textuales. El objetivo funcional es superar una búsqueda exacta por palabras clave: el usuario puede formular restricciones como “sin horno”, preferencias dietéticas, peticiones de recetas rápidas o solicitudes de resumen de opiniones.

Desde el punto de vista de la asignatura, el sistema integra las técnicas exigidas por el enunciado:

- **Embeddings**: generación de representaciones densas con **SentenceTransformer**.
- **Modelado de tópicos**: entrenamiento offline con BERTopic sobre el corpus de recetas.
- **Resumen extractivo**: extracción de frases centrales con TextRank sobre reseñas.
- **LLM**: generación final de respuestas naturales mediante un modelo local.
- **RAG**: recuperación de documentos locales e inyección de contexto al prompt.
- **Agentes**: router que decide entre recomendación local, resumen y búsqueda web.
- **Adquisición y preprocesado**: limpieza y filtrado de corpus como extra opcional.

El proyecto evita frameworks completos de alto nivel como LangChain o LlamaIndex. La recuperación, el enrutamiento, el resumen y la construcción del prompt están implementados directamente en módulos Python del repositorio.

2. Diseño Arquitectónico Global

La arquitectura se divide en dos bloques: un **pipeline offline**, que prepara el corpus y los índices persistentes, y un **pipeline online**, que atiende las consultas del usuario en tiempo real.

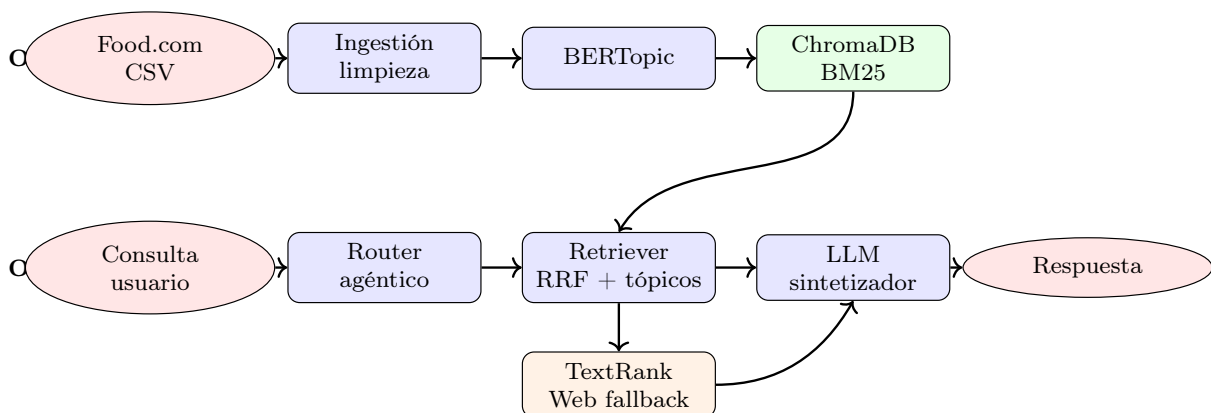


Figura 1: Arquitectura global de CulinaryRAG.

2.1. Pipeline Offline

El pipeline offline concentra las operaciones costosas que no deben ejecutarse en cada consulta:

1. **Preprocesado** (`datasets/preprocesado.py`): carga los CSV originales, conserva campos relevantes, elimina reseñas nulas, filtra usuarios con al menos 8 reseñas y cruza interacciones con recetas válidas.
2. **Modelado de tópicos** (`modules/topics.py`): entrena BERTopic sobre una muestra estratificada por tiempo de preparación. El modelo puede usar semillas culinarias para mejorar la interpretabilidad de los clústeres.
3. **Indexación semántica y léxica** (`offline_pipeline.py`, `modules/retriever.py`): crea embeddings a partir de nombre, descripción e ingredientes, los persiste en ChromaDB y genera un índice BM25 serializado.

2.2. Pipeline Online

El pipeline online se ejecuta desde `main.py`. El flujo es:

1. Carga de índices locales, modelo de tópicos si existe, interacciones y LLM.
2. Enrutamiento de la consulta mediante `modules/agent.py`. El router intenta clasificar con el LLM y valida la salida contra acciones permitidas; si no hay LLM o la salida no es válida, aplica una heurística determinista.
3. Ejecución de herramienta: recomendación local, resumen de reseñas o búsqueda web simple.
4. Construcción del prompt con el contexto recuperado.
5. Síntesis final con el LLM configurado.

3. Matriz de Cumplimiento del Enunciado

Requisito	Implementación en el proyecto	Evidencia
Embeddings	Representaciones vectoriales de recetas mediante <code>paraphrase-multilingual-MiniLM-L12-v2</code> .	<code>retriever.py</code> : indexación
Modelado de tópicos	BERTopic entrenado offline, con muestreo estratificado y opción de semillas.	<code>modules/topics.py</code> , <code>modules/experimento.py</code>
Resumen extractivo	TextRank propio con matriz de similitud y PageRank para extraer frases centrales de reseñas.	<code>modules/summarizer.py</code>
LLM	Carga de modelo local mediante Transformers y generación determinista.	<code>modules/llm.py</code>
RAG	Recuperación híbrida local y prompt final con contexto recuperado.	<code>main.py</code> , <code>build_prompt()</code>
Agentes	Router de herramientas: recomendar, resumir o web; ejecución modular de acciones.	<code>modules/agent.py</code>
Arquitectura offline/online	Separación entre <code>offline_pipeline.py</code> y el bucle interactivo <code>main.py</code> .	Figura 1
Extras opcionales	Limpieza, filtrado de calidad, query expansion, BM25, RRF, Cross-Encoder y fallback web.	<code>datasets/</code> , <code>modules/retriever.py</code>

Cuadro 1: Trazabilidad entre requisitos del enunciado e implementación.

4. Detalles de Implementación

4.1. Ingestión y Limpieza

La construcción del corpus se implementa en `datasets/preprocesado.py`. El filtrado elimina interacciones sin reseña escrita y conserva usuarios con volumen suficiente de opiniones, reduciendo ruido y favoreciendo que el módulo de resumen tenga texto útil.

```
1 df_interactions = df_interactions.dropna(subset=['review'])
2 reviews_por_user = df_interactions['user_id'].value_counts()
3 usuarios_def = reviews_por_user[reviews_por_user >= 8].index
4 df_interactions_filtrado = df_interactions[
5     df_interactions['user_id'].isin(usuarios_def)
6 ].copy()
```

Listing 1: Filtro de interacciones de calidad.

4.2. Recuperación Híbrida

El módulo `modules/retriever.py` combina dos señales complementarias:

- **ChromaDB + embeddings**: captura similitud semántica aunque no coincidan literalmente las palabras.
- **BM25**: preserva precisión léxica para ingredientes, etiquetas o restricciones concretas.

Los rankings se fusionan mediante *Reciprocal Rank Fusion* con $k = 60$. Si existe modelo BERTopic cargado, se aplica un bonus del 50 % cuando el tópico del documento coincide con el de la consulta.

```
1 score = 0.0
2 if doc_id in rank_sem:
3     score += 1.0 / (RRF_K + rank_sem[doc_id])
4 if doc_id in rank_lex:
5     score += 1.0 / (RRF_K + rank_lex[doc_id])
6 if query_topic != -1 and id_to_topic.get(doc_id) == query_topic:
7     score *= 1.5
```

Listing 2: Fusión RRF con refuerzo por tópico.

Después de la fusión, un Cross-Encoder (`cross-encoder/ms-marco-MiniLM-L-6-v2`) reordena los candidatos. Además, la función `_constraint_adjustment()` penaliza recetas incompatibles con restricciones detectadas, por ejemplo recetas con horno cuando la consulta pide “sin horno”.

4.3. Modelado de Tópicos

BERTopic se entrena con embeddings, reducción UMAP y clustering HDBSCAN. El corpus usado para tópicos concatena nombre y etiquetas de receta, de forma que los clústeres reflejen familias culinarias. El muestreo estratificado por cuartiles de `minutes` evita que el entrenamiento quede dominado por recetas de una sola duración.

4.4. Resumen Extractivo

`modules/summarizer.py` implementa TextRank sin depender de servicios externos. El procedimiento divide reseñas en oraciones, tokeniza, construye una matriz de similitud de coseno y aplica PageRank. Las frases con mayor centralidad se devuelven en el orden original para conservar legibilidad.

4.5. LLM y Generación

`modules/llm.py` encapsula la carga de modelos compatibles con Transformers. Por defecto, `main.py` intenta usar una copia local de Gemma 3 1B Instruct; si no está disponible, usa Qwen/Qwen2.5-1.5B-Instruct. La generación se realiza con `do_sample=False`, lo que mejora la reproducibilidad durante la defensa.

5. Validación y Discusión

5.1. Validación por Módulos

Módulo	Qué se valida	Criterio de aceptación
Preprocesado	CSV procesados sin campos críticos nulos y con reseñas utilizables.	Se generan <code>Processed_recipes.csv</code> y <code>Processed_interactions.csv</code> .
Tópicos	Coherencia e interpretabilidad de clústeres BERTopic.	Tópicos no triviales, outliers controlados y etiquetas comprensibles.
Retriever	Calidad del ranking ante consultas con ingredientes y restricciones.	Los primeros resultados respetan intención y restricciones principales.
Resumen	Extracción de frases centrales de reseñas.	El resumen conserva opiniones representativas sin saturar el prompt.
Agente	Selección correcta de herramienta.	Recomendación para consultas culinarias, resumen si se piden reseñas, web si se pide actualidad.
LLM/RAG	Respuesta final en español basada en contexto local.	La respuesta cita recetas recuperadas y no inventa recetas fuera del contexto.

Cuadro 2: Plan de validación funcional por módulo.

5.2. Experimento de Tópicos

El script `modules/experimento.py` permite comparar un BERTopic no supervisado frente a un modelo guiado con semillas. La hipótesis es que las semillas producen clústeres más alineados con categorías culinarias útiles para recomendación.

Aspecto evaluado	Modelo sin semillas	Modelo con semillas
Formación de tópicos	Agrupación puramente estadística.	Agrupación guiada por categorías culinarias.
Interpretabilidad	Puede mezclar ingredientes frecuentes.	Más cercana a conceptos como postres, vegano o desayuno.
Uso en el retriever	Menos estable para topic boost.	Más útil para reforzar recetas del mismo dominio.
Coste de preparación	Menor intervención humana.	Requiere definir semillas, pero mejora control semántico.

Cuadro 3: Comparación cualitativa del modelado de tópicos.

5.3. Limitaciones

El sistema cumple el enunciado, pero presenta límites relevantes:

- El fallback web usa una consulta HTML básica a DuckDuckGo; sirve como demostración agéntica, no como scraper robusto.

- El Cross-Encoder usado está entrenado para inglés, por lo que el sistema traduce o expande consultas españolas con términos ingleses para mejorar la señal.
- La calidad final depende de haber ejecutado previamente el pipeline offline y de disponer de los modelos descargados.

6. Instrucciones de Ejecución

El flujo recomendado para reproducir el sistema es:

1. Instalar dependencias con `pip install -r requirements.txt`.
2. Colocar los CSV originales en `datasets/original/`.
3. Ejecutar `python datasets/preprocesado.py`.
4. Entrenar tópicos con `python modules/topics.py`.
5. Crear índices con `python offline_pipeline.py`.
6. Lanzar el asistente con `python main.py`.

Para validar únicamente la recuperación se puede ejecutar `python test_search.py`, lo que permite probar el ranking sin cargar todo el LLM.

7. Conclusiones y Trabajo Futuro

CulinaryRAG demuestra una integración completa de técnicas de minería de textos en un asistente especializado. El sistema separa correctamente preparación offline y ejecución online, combina recuperación semántica y léxica, usa tópicos para mejorar el ranking, resume reseñas con TextRank y genera respuestas finales mediante un LLM.

Como trabajo futuro se propone:

1. Añadir perfiles de usuario persistentes para personalizar los pesos de ranking.
2. Reemplazar el fallback web por un extractor estructurado con control de fuentes.
3. Incorporar métricas cuantitativas guardadas automáticamente para comparar variantes de recuperación.
4. Implementar una restricción real de logits en el router si se quiere garantizar selección de herramienta a nivel de token.